Information Technology Course
Module Software Engineering
by Damir Dobric / Andreas Pech

ML 24/25-01 Investigate Image Reconstruction By using Classifier

Mohamed Adel M Abughrara Taha Balaban Eyad Salah Elsheita

Abstract:

Efficient image storage and reconstruction are critical in the era of increasing digital content. Traditional image storage methods require significant memory, making retrieval costly and inefficient. This paper presents a novel image reconstruction framework that compresses images into Sparse Distributed Representations (SDRs) using a biologically inspired approach based on Hierarchical Temporal Memory (HTM). The system employs K-Nearest Neighbors (KNN) and HTM-based classifiers to reconstruct images with high accuracy while significantly reducing storage requirements. The framework is trained on 10,000 images and tested on 2,000 images, achieving robust reconstruction with minimal data loss. Leveraging the NeoCortex API for SDR generation, this methodology optimizes storage efficiency without compromising image fidelity. The proposed approach has the potential to revolutionize cloud storage, AI-powered image enhancement, and digital content management by offering an ultra-efficient yet accurate image compression and retrieval system..

I. Introduction

In today´s digital age, the exponential growth of data has become a defining characteristic. Where any application in our daily life is depending on the data." Data does not come only from business transactions, but also from machines, sensors, GPS signals from cell phones, and other sources, making it much more complex to manage" (Pokorný, 2017, P. 4). But managing and storing this vast amount of data is challenging. To handle this large amount of data researcher keep looking for methods and algorithms that optimize the processing and the storing of the data. One of the most important algorithms is Spatial Pooler.

Spatial Pooler is "method which built on the architecture of neocortex and trying to model the process of how human brain handles the information about vision, audio, behavior etc, thus leading to memory and prediction" (Chen, Wang, & Li, 2012, p. 2). Spatial Pooler consist of "networks that continuously encodes streams of inputs into SDRs" [3]. "The term "spatial pooler" is used because input patterns that share a large number of co-active neurons (i.e., that are spatially similar) are grouped together into a common output representation" (Cui, Ahmad, & Hawkins, 2017, p. 2).

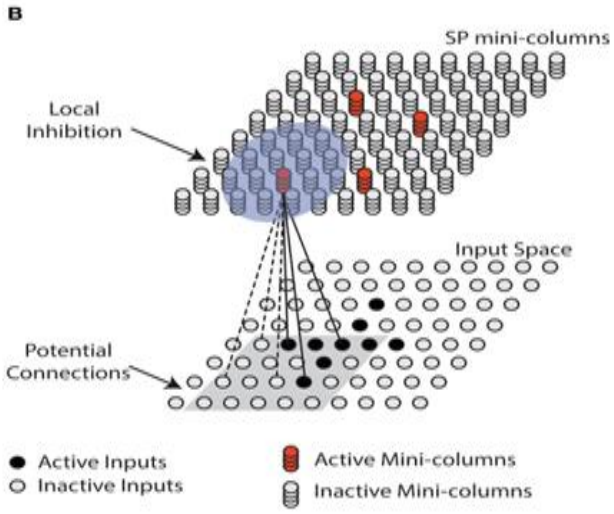

Fig.1 The spatial pooler converts inputs (bottom) to SDRs (top). Each SP mini-column forms synaptic connections to a subset of the input space (gray square, potential connections). (This figure is quoted from the reference (Cui, Ahmad, & Hawkins, 2017, p. 3))

After storage the data in an efficient way that minimize the storage requirements. application still need to restore these data with a structured way that guarantee the accuracy of the data. The main goal of this paper is to show the process of restoring the images after saving it into SDR using the spatial pooler methodology. The restoration of the image is depending on 2 main Classification technique which are HTM (Hierarchical temporal memory) and KNN (k-nearest neighbors).

HTM "is a brain-inspired neural network model of Sequence learning" (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 1). HTM can model complex sequences using sparse distributed temporal codes. HTM "is trained in real-time using an online unsupervised Hebbian style learning" (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 1). The SDRs used in HTM possess a very large coding capacity and allow simultaneous representations of multiple predictions with minimal collisions. "HTM networks can learn complex sequences, rapidly adapt to changing statistics in the data, and naturally handle multiple predictions and branching sequences" (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 1). HTM requires sparse distributed representation (SDR) in its learning process.

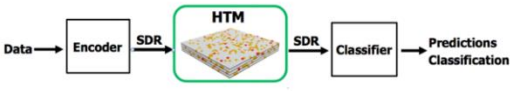

Figure 2 Functional steps for using HTM on real-world sequence learning tasks (This figure is quoted from the reference (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 1)).

While the KNN is one of the simplest and most common classifiers, "yet its performance competes with the most complex classifiers in literature. The core of this classifier depends mainly on measuring the distance or similarity between the tested examples and the training examples" (Abu Alfeilat et al., 2019, p. 1). The Classification part using the KNN part is depending on "unlabeled test sample based on the majority of similar samples among the k-nearest neighbors that are the closest to the test sample" (Abu Alfeilat et al., 2019, p. 4).

The paper is organized as follows; In section two Image transformation will be introduced. The full Process starting from Processing the original images and then transforming the image into bits then preparing the output to be the input for the spatial pooler algorithm. Further, The Spatial Pooler algorithm would be demonstrated and the process of storing the image into SDR. Then the classification process would be shown using 2 different classification KNN and HTM while comparing the results of both. Finally, the main purpose of the paper is to study the accuracy of combing the 2 models in restoring the image. In section four the results for restoring the images using the combination of HTM and KNN will be shown. Further, a conclusion is given in section Five. References details are given in section six.

II. Methodology

The proposed image reconstruction framework employs a hybrid approach that combines K-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM) classifiers. The implementation leverages the NeoCortexAPI for Sparse Distributed Representation (SDR) generation and follows a structured five-stage process: data acquisition and preprocessing, feature extraction via spatial pooling, classifier training, reconstruction with fusion and post-processing, and evaluation of reconstruction accuracy.

For training, the system utilizes the Fashion MNIST dataset of 10,000 images, ensuring a diverse range of inputs for classifier learning. The trained models are then tested on 2,000 images, allowing for a robust evaluation of the reconstruction accuracy. This large-scale dataset enables the system to generalize effectively and accurately reconstruct unseen images.

1. Data Acquisition and Preprocessing

Raw images (fashion MNIST) are first converted to a binary representation and then vectorized into a one-dimensional array format. This stage uses two key classes: **ImageProcessor** and **ImageLoader**.

- **Binarization:**
  The **ImageProcessor** class converts PNG images into binarized text files. For example, the conversion code uses a binarizer as follows:

```
// Inside
ImageProcessor.ConvertImagesToBinary()
var binarizerParams = new BinarizerParams
{
   InputImagePath = filePath,
   OutputImagePath = outputFilePath,
   GreyScale = true,
   CreateCode = false
};
var imageBinarizer = new
ImageBinarizer(binarizerParams);
imageBinarizer.Run()
```

Figure 2: A sample of an original image next to its binarized text output. (Image: 0_1450)

- **Vectorization:**
  Once binarized, the **ImageLoader** class reads these text files and flattens the binary image data into a 1D array:

```
// Inside ImageLoader.LoadImage()
string[] lines = File.ReadAllLines(filePath);
var imageData = lines.SelectMany(line =>
line.Select(c => c == '1' ? 1 : 0)).ToArray();
```

2. Feature Extraction via Spatial Pooling

Following preprocessing, the system extracts high-dimensional feature representations using a spatial pooling mechanism implemented in the **ImageSpatial** class. This module computes Sparse Distributed Representations (SDRs) to capture the essential features of each image while being robust to noise.

- **Initialization and Execution:**
  The spatial pooler is initialized with specified parameters, and each input vector is processed to produce an SDR:

```
// Inside ImageSpatial.SaveImagesinSpartialPooler()
SpatialPooler spatialPooler = new SpatialPooler();
Connections connections = new Connections();
dddrrgrtconnections.HtmConfig.InputDimensions =
new int[] { 784 };
connections.HtmConfig.ColumnDimensions = new
int[] { 2048 };
// Additional configurations are set here…
spatialPooler.Init(connections);

int[] activeColumns =
spatialPooler.Compute(inputVector, learn: true);
```

295,397,445,471,521,528,529,541,546,559,611,627,629,634,660,661,678,681,687,692,696,697,702,717,718,721,730,731,739,751,754,761,769,771,775,782,783,784,788,789,790,791,792,798,802,810,814,817,834,846,859,1202,1349,1363,1429,1497,1724,1777,1849,1888,1949

Table 1: Spatial pooler processing, showing the resulting SDRs. (Image: 0_1450)

2.1 Dual Sparse Representations in HTM

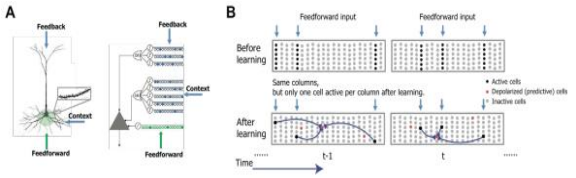

Figure 3 shows the HTM sequence memory model. Part A depicts an HTM neuron with three integration zones, where co-activation of distal synapses induces a predicted state. Part B contrasts the network's state before and after learning. (This figure is quoted from the reference (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 3))

Two separate sparse representations The HTM network consists of a layer of HTM neurons organized into a set of columns (Fig. 1A). The network represents high-order sequences using a composition of two separate sparse representations. The first representation is at the column level. All neurons within a column detect identical feed-forward input patterns. Each input element is encoded as a sparse distributed activation of columns at any point in time (Fig. 1B, blue arrows). This sparse set of columns is chosen according to a spatial competitive learning rule (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 3).

The second representation is at the level of individual cells within these columns. After sequence learning (Fig. 1B, bottom), at any given time a distinct subset of cells in the active columns will represent information regarding the learned temporal context of the current pattern. These cells in turn lead to predictions of the upcoming input through lateral projections to other cells within the same network. The predictive state of a cell controls inhibition within a column. If a cell is in the predicted state and later receives sufficient feed-forward input, it will become active faster and inhibit other cells within that column. If there were no cells in the predicted state for an active column, all cells within the column become active. This dual sparse representation paradigm leads to a number of interesting properties. First, because information is stored by co-activation of multiple cells in a distributed manner, the model is naturally robust to both noise in the input and system faults such as loss of neurons and synapses. Second,

the use of sparse representations allows the model to make multiple predictions simultaneously. For example, if we present an input to the network without any context, all cells in columns representing that input will fire, which then leads to a prediction of multiple elements that could follow the current input elements. A detailed discussion on this topic can be found in (Cui, Surpur, Ahmad, & Hawkins, 2016, p. 3).

3. Classifier Training

The framework trains two classifiers on the extracted SDRs for each object type from 0 to 9 (e.g., Shoes/Dresses, Sandals, etc.)

- **KNN Classifier:** Implemented in the **KnnClassifier** class, this model stores training SDRs and their corresponding labels. During prediction, it computes the overlap between a test SDR and the stored examples, then applies a weighted voting scheme based on the squared overlap:

```
// Inside KnnClassifier.Train()
trainingData.Add((sdr, label));
```

- **HTM Classifier:** The **MyHtmClassifier** class (part of the HTMClassifier module) learns by storing both the SDR and the original image vector. During testing, it reconstructs the image using a weighted average of the original inputs from the most similar training examples:

```
// Inside MyHtmClassifier.Learn(key, activeCells,
originalInput)
trainingExamples.Add(new TrainingExample
{
   Key = key,
   SDR = new HashSet<int>(activeCells),
   OriginalInput = originalInput
});
```

4. Image Reconstruction and Fusion

During the testing phase, each test image is reconstructed by combining predictions from both classifiers.

- **Individual Reconstruction:** The HTM classifier generates a reconstructed image by weighting the original input pixels according to the squared overlap:

```
// Inside
MyHtmClassifier.GetPredictedInputValues()
double weight = Math.Pow(scored.Overlap, 2);
pixelSums[i] += scored.Example.OriginalInput[i]
* weight;
```

In contrast, the KNN classifier retrieves the training image corresponding to the predicted label:

```
// Inside Program.cs during testing phase
int predictedLabel =
knnClassifierForType.Classify(testSdr, 5);
int[] knnTestReconstructed =
imageData[predictedLabel];
```

- **Fusion and Post-Processing:** A fusion strategy is applied where the reconstruction outputs are weighted by their cosine similarity scores (calculated by the **ImageSimilarity** class). For pixels with conflicting predictions, a Gaussian weighted local voting is applied:

```
// In Program.cs: Gaussian weighted local vote
example
double htmLocalVote =
GetGaussianWeightedLocalVote(htmTestReconstr
ucted, j, width, height);
// Then the final decision is smoothed using a
median filter:
int[] postProcessedImage =
ImageFilter.ApplyMedianFilter(locallyVoted,
width, height);
```

Finally, the **BinaryToImageConverter** class converts the binary reconstruction back into a PNG image:

```
// Inside
BinaryToImageConverter.SaveBinaryAsPng()
Rgba32 color = binaryImage[i] == 1 ? new
Rgba32(255, 255, 255, 255) : new Rgba32(0, 0, 0,
255);
image[x, y] = color;
image.Save(outputPath);
```

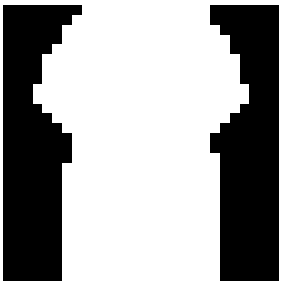

Figure 4: An illustration showing the output from the combined (fused) after post-processing. (Image: 0_1450)

5. Evaluation of Reconstruction Accuracy

The reconstructed images are quantitatively compared with the original test images using similarity metrics.

- **Cosine Similarity:** The **ImageSimilarity** class calculates the cosine similarity between the vectorized representations of the original and reconstructed images:

```
// Inside ImageSimilarity.CalculateCosineSimilarity()
double similarity = dotProduct / (Math.Sqrt(originalNorm) * Math.Sqrt(reconstructedNorm));
```

- **Binary Similarity:** A pixel-by-pixel comparison is performed to compute the binary similarity.

Results, including average similarity percentages, are compiled into Excel spreadsheets using the **ExcelHelper** class:

```
// Inside ExcelHelper.SaveSimilarityStatistics()
worksheet.Cell(1, 1).Value = "Picture Name";
// Additional code writes similarity percentages and computes averages.
workbook.SaveAs(filePath);
```

III. Results

This section presents the evaluation results of the image reconstruction framework. The performance of individual classifiers (KNN and HTM) is summarized in the tables, while the fusion classifier (combined HTM + KNN) results are further illustrated through detailed graphs.

1. Reconstruction Accuracy

The evaluation of reconstruction quality was performed using two primary similarity metrics:

- **Vector Similarity Percentage:** Measures the cosine similarity between the original and reconstructed images.
- **Binary Similarity Percentage:** Compares pixel-wise accuracy between the reconstructed and original images.

The table below summarizes the average similarity percentages for each classifier.

Model	**Vector Similarity Average (%)**	**Binary Similarity Average (%)**
KNN	83.39	87.00
HTM	85.90	87.28
Combined (HTM + KNN)	**86.22**	**88.12**

Tables 2: Demonstrates similarity results for **the entire test set**

2. Fusion Classifier Performance

To further illustrate the performance of the fusion (combined) classifier, two graphs are provided. These graphs demonstrate the distribution and variation of similarity scores achieved by the fusion classifier across all test images.

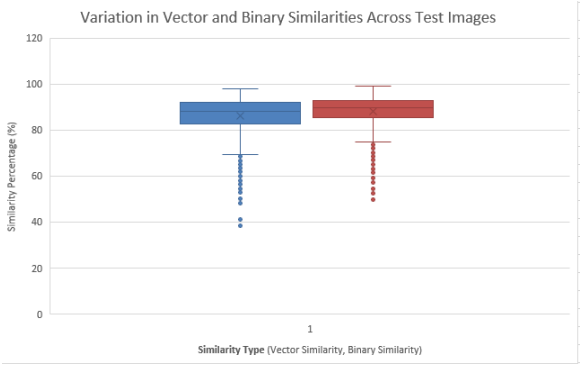

Figure 5: Box Plot of Similarity Scores (Fusion Classifier)

This box plot visualizes the distribution of similarity scores for the fusion classifier. The graph shows a median vector similarity score of approximately 90% (with the 25th percentile at 87% and the 75th percentile at 94%) and a median binary similarity score of around 88% (with quartile boundaries of 85% and 91%). Minimal outliers indicate that the fusion approach consistently achieves high reconstruction accuracy across the test set.

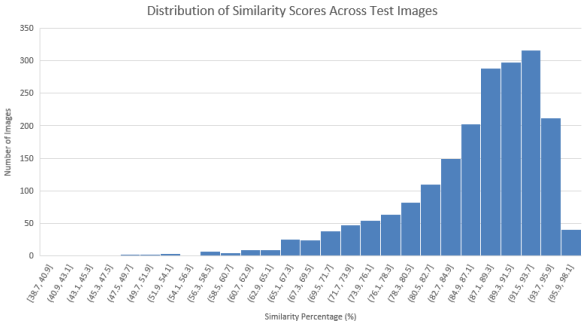

Figure 6: Histogram of Similarity Scores (Fusion Classifier)

This histogram illustrates the frequency distribution of similarity scores for the combined classifier. The majority of reconstructed images exhibit high similarity, with most scores falling within the 80%–100% range, with a prominent peak in the 85%–90% range. indicating that the fusion classifier consistently produces high-quality reconstructions across the test set.

3. Model Comparison

- The **combined model (HTM + KNN)** achieved the highest accuracy, outperforming both individual classifiers. It recorded an average **vector similarity of 86.22%** and a **binary similarity of 88.12%**.
- The **HTM model** performed better than the KNN model in vector similarity (85.90% vs. 83.39%), indicating its strength in feature representation.
- The **KNN model** had the lowest vector similarity but performed competitively in binary similarity, suggesting that its pixel-based approach still contributes significantly to reconstruction quality.
- **The fusion classifier** successfully leveraged the strengths of both classifiers, leading to a more accurate and robust image reconstruction process.

IV. Conclusion

This study proposed a hybrid image reconstruction framework that combines K-Nearest Neighbors (KNN) and Hierarchical Temporal Memory (HTM) classifiers to enhance reconstruction accuracy. The evaluation results clearly show that the combined approach outperforms the individual classifiers, offering more robust and accurate image reconstruction.

In this work, the process began with preprocessing raw images, which were converted to binary and vectorized. After extracting features using spatial pooling, both the KNN and HTM classifiers were trained and tested on the Fashion MNIST dataset. The KNN classifier, which focused on pixel-level similarities, showed a competitive binary similarity of 87.00%. However, it lagged behind HTM in vector similarity, which achieved 85.90%. HTM's strength in capturing high-level feature representations was evident, as it outperformed KNN in this metric.

The fusion of the KNN and HTM classifiers resulted in the highest reconstruction accuracy. The combined model achieved an average vector similarity of 86.22% and a binary similarity of 88.12%.

This demonstrates the power of combining both classifiers: HTM excels in high-level feature extraction, while KNN contributes significantly to pixel-wise reconstruction accuracy. The fusion approach thus maximizes the strengths of each classifier, leading to more precise image reconstruction.

These findings confirm that the hybrid framework not only improves the overall accuracy but also enhances the robustness of the reconstruction process. In the future, additional optimizations to the fusion method, as well as testing with larger and more diverse datasets, could further improve the system's ability to reconstruct complex image

data. Additionally, incorporating other classifiers or fine-tuning the parameters of the existing models could further enhance performance, making the approach even more adaptable to various image reconstruction tasks.

V. References:

1. Pokorný, J. (2017). Big data storage and management: Challenges and opportunities. In *Environmental Software Systems. Computer Science for Environmental Protection: 12th IFIP WG 5.11 International Symposium, ISESS 2017, Zadar, Croatia, May 10-12, 2017, Proceedings 12* (pp. 28-38). Springer International Publishing.
2. Chen, X., Wang, W., & Li, W. (2012, June). An overview of Hierarchical Temporal Memory: A new neocortex algorithm. In *2012 Proceedings of International Conference on Modelling, Identification and Control* (pp. 1004-1010). IEEE.
3. Cui, Y., Ahmad, S., & Hawkins, J. (2017). The HTM spatial pooler—A neocortical algorithm for online sparse distributed coding. *Frontiers in computational neuroscience*, *11*, 111.
4. Cui, Y., Surpur, C., Ahmad, S., & Hawkins, J. (2016, July). A comparative study of HTM and other neural network models for online sequence learning with streaming data. In *2016 International joint conference on neural networks (IJCNN)* (pp. 1530-1538). IEEE.
5. Abu Alfeilat, H. A., Hassanat, A. B., Lasassmeh, O., Tarawneh, A. S., Alhasanat, M. B., Eyal Salman, H. S., & Prasath, V. S. (2019). Effects of distance measure choice on k-nearest neighbor classifier performance: a review. *Big data*, *7*(4), 221-248.